GraphRAG for Supply Chain Risk Intelligence

Mounia Baddou **Elghali Benjelloun** **Khaoula Mejhoudi**

Supervised by Pr. Lamiae Azizi

College of Computing - Generative AI Course, Spring 2026

April 2026

Contents

1 Introduction and Motivation **2**
1.1 Why Standard RAG Falls Short . . . 2
1.2 Existing Approaches and Their Limitations . . . 2
1.3 Contributions . . . 2

2 Related Work **3**
2.1 GraphRAG Foundations . . . 3
2.2 Supply Chain Knowledge Graphs . . . 3
2.3 Graph Neural Networks for Supply Chain . . . 3
2.4 Disruption Propagation Models . . . 3

3 System Architecture **3**
3.1 Data Foundation . . . 4

4 Technical Deep-Dive **4**
4.1 Module 1: Knowledge Graph Construction . . . 4
4.1.1 Triple Extraction via LLM . . . 4
4.1.2 Entity Canonicalization . . . 5
4.1.3 Graph Construction and Edge Weights . . . 5
4.1.4 Node Embeddings and FAISS Indexing . . . 5
4.2 Module 2: GraphRAG Retrieval Layer . . . 6
4.2.1 Dual-Stage Retrieval . . . 6
4.2.2 Score Fusion . . . 6
4.2.3 Query Routing: Local vs. Global . . . 6
4.3 Module 3: Disruption Simulation Engine . . . 7
4.3.1 Propagation Model Selection . . . 7
4.3.2 Attenuated Bottleneck Routing . . . 7
4.3.3 Score Merging . . . 8
4.4 Module 4: LLM Generation and Risk Reporting . . . 8
4.4.1 Context Assembly . . . 8
4.4.2 Model Choice: Claude Sonnet 4.6 . . . 8
4.4.3 Structured Output . . . 8
4.4.4 Semantic Query Cache . . . 8

5 Evaluation and Results **9**
5.1 Experimental Setup . . . 9
5.2 Results . . . 9

6 Limitations and Future Work **9**
6.1 Current Limitations . . . 9
6.2 Future Directions . . . 10

7 Conclusion **10**

A Live System Output: Worked Example **12**

1 Introduction and Motivation

Modern supply chains are complex, non-linear webs of interdependencies spanning dozens of countries, hundreds of companies, and thousands of components. A single disruption event (a port closure, a factory fire, or a geopolitical sanction) can cascade through these networks in ways that are invisible to conventional monitoring tools. The challenge is not a lack of data: companies report extensively on their supplier relationships in earnings calls, SEC filings, and procurement disclosures. The challenge is structural: the relevant information lives in *relationships between entities*, not in any individual document.

1.1 Why Standard RAG Falls Short

The dominant paradigm for grounding large language model (LLM) outputs in domain-specific data is Retrieval-Augmented Generation (RAG) [1]. Standard RAG encodes documents as dense vectors and retrieves the most semantically similar chunks at query time. For supply chain risk this is fundamentally insufficient for three reasons:

(i) **Relationships matter more than content.** Knowing that "TSMC is a semiconductor manufacturer in Taiwan" is far less actionable than tracing the full chain Apple → TSMC → ASML → Taiwan (earthquake risk). Semantic similarity retrieval cannot reconstruct this chain.

(ii) **Multi-hop reasoning is required.** A tier-3 supplier disruption cannot be surfaced by any amount of keyword or embedding search; it requires traversing multiple edges in a dependency graph.

(iii) **Global queries are unanswerable locally.** Questions such as "What is our overall exposure to East Asian geopolitical risk?" demand reasoning over the *entire* corpus. Edge et al. (2024) demonstrate that vanilla RAG fails completely on this class of query [1].

1.2 Existing Approaches and Their Limitations

Three alternative approaches are commonly considered:

- **Massive context prompting.** Feeding entire corpora into the LLM context window. Irrelevant context drowns the signal and scales poorly.
- **Standard RAG.** As described above: efficient but relationship-blind.
- **LLM fine-tuning.** Encodes knowledge into model weights. Expensive to retrain and static once deployed.

None of these approaches natively model the *graph structure* that is intrinsic to supply chains. This project addresses that gap.

1.3 Contributions

This work presents an end-to-end **GraphRAG pipeline for supply chain risk intelligence** with the following contributions:

(1) A fully automated knowledge graph construction pipeline that extracts typed supply chain triples from unstructured earnings call text using open-source LLMs.

(2) A dual-stage retrieval layer combining semantic seed search with K-hop graph expansion.

(3) **Attenuated Bottleneck Routing (ABR)**, a disruption propagation algorithm that models how shocks cascade through supplier dependency networks.

(4) A semantic query cache that eliminates redundant LLM calls on equivalent queries.

2 Related Work

2.1 GraphRAG Foundations

Edge et al. (2024) introduced the GraphRAG framework, which augments retrieval with Louvain community detection and BART-generated community summaries to support both local (entity-specific) and global (theme-level) queries [1]. Their work motivates the distinction between query types that we implement in our retrieval routing layer. Peng et al. (2024) provide a taxonomy of GraphRAG approaches under the G-Indexing / G-Retrieval / G-Generation framework, which we use to position our system [2].

2.2 Supply Chain Knowledge Graphs

Almahri et al. (2024) demonstrate LLM-driven named entity recognition and relation extraction for supply chain knowledge graph construction [3]. Their prompting strategy for typed triple extraction directly informs our extraction prompt design. Kosasih et al. (2024) explore neurosymbolic approaches to supply chain risk, introducing the concept of edge criticality weighting that we operationalize as our redundancy-based edge weight formula [4].

2.3 Graph Neural Networks for Supply Chain

Wasi et al. (2024) establish a benchmark taxonomy for GNN-based supply chain tasks and introduce the SupplyGraph dataset [5]. Their evaluation framing (multi-hop accuracy, disruption coverage, answer completeness) directly shapes our metrics.

2.4 Disruption Propagation Models

Our disruption engine draws theoretical grounding from recent work on pressure wave propagation models in supply networks (MDPI, 2026) [6]. That work establishes the physical analogy between pressure attenuation in fluid networks and risk dissipation across supplier tiers, which is the key insight behind the Attenuated Bottleneck Routing formulation.

3 System Architecture

The system is organized as a five-stage pipeline, depicted in Figure 1.

```
Raw Corpus (S&P 500 earnings call transcripts)
      |
      v
[Module 1] Knowledge Graph Construction
   LLM triple extraction -> canonicalization -> NetworkX DiGraph
   BGE-large-en-v1.5 node embeddings -> FAISS IndexFlatIP
      |
      v
[Module 2] GraphRAG Retrieval Layer
   FAISS semantic seed search -> K-hop ego-graph expansion
   Score fusion: 0.6 * semantic + 0.4 * degree_centrality
   Louvain community summaries (BART) for global queries
      |
      v
[Module 3] Disruption Simulation Engine
   Attenuated Bottleneck Routing: F_v = min(F_u, C_{u,v}) * gamma
   Modified Dijkstra (max-heap) -> path traces -> severity tiers
   Score merging: 0.5 * retrieval + 0.5 * disruption
      |
      v
[Module 4] LLM Generation & Risk Reporting
   Claude Sonnet 4.6 -> structured 6-section XML report
   Semantic query cache (BGE + FAISS, threshold 0.92)
      |
      v
[Module 5] Evaluation & Ablation Study
   Baseline LLM vs. Vector RAG vs. GraphRAG+Simulation
```

Figure 1: End-to-end pipeline architecture.

The pipeline ingests raw supply chain text and outputs a structured, graph-grounded risk report. Crucially, the LLM at generation time is *not* asked to perform relational reasoning; that work is done upstream by the graph and the simulation engine. The model's role is interpretation and communication.

3.1 Data Foundation

The corpus consists of S&P 500 earnings call transcripts keyword-filtered for supply chain content. Each record carries a source company, sector, ticker, year, quarter, and raw text (4,800 words per company). The S&P 500 was chosen for three reasons: (i) it provides cross-sector coverage, embedding cross-domain dependencies natively; (ii) scraping individual companies does not scale to a representative global network; and (iii) the corpus is publicly available without licensing constraints. The resulting graph contains **7,292 nodes** and **10,820 edges** stored as a NetworkX `DiGraph`.

4 Technical Deep-Dive

4.1 Module 1: Knowledge Graph Construction

4.1.1 Triple Extraction via LLM

Each document is split into overlapping 2,400-character chunks at sentence boundaries. Every chunk is sent to an LLM which must return a list of structured JSON triples of the form:

```
{
  "head": "TSMC", "head_type": "Supplier",
```

```
  "relation": "supplies","tail": "Apple",
  "tail_type": "Manufacturer",
  "confidence": 0.92,
  "evidence": "TSMC␣supplies␣advanced␣chips␣to␣Apple,␣NVIDIA,␣and␣AMD"
}
```

Why an LLM for extraction rather than a rule-based NER model? Earnings call text is informal and deeply implicit. A sentence such as *"our single-source arrangement with a Taiwanese fab puts us in a difficult position"* requires domain understanding to map to a `depends_on` relation with high criticality. Rule-based parsers cannot handle this; general-purpose NER models extract entities but not typed, directional relationships. An LLM is the practical choice for this text type.

Model choice: Qwen2.5-32B (via Ollama) / Llama-3.3-70B (via Groq). Qwen2.5-32B was selected as the primary local extraction model because it offers a strong balance between instruction-following quality and deployability on consumer-grade hardware. At 32 billion parameters it is large enough to handle implicit domain reasoning without requiring the infrastructure of a 70B model. Llama-3.3-70B via Groq serves as a cloud-based alternative when extraction quality must take precedence over compute cost.

The ontology is intentionally small and fixed: entity types `{Supplier, Manufacturer, Part, Port, Region, DisruptionEvent, Customer, LogisticsRoute}` and relation types `{supplies, depends_on, ships_through, located_in, affected_by, alternative_to, sells_to, connects}`. Constraining the schema forces the model to map free-form text onto a consistent vocabulary rather than generating incompatible free-form triples across chunks.

4.1.2 Entity Canonicalization

After extraction, the entity registry contains aliases: "Taiwan Semiconductor", "TSMC Inc.", and "Taiwan Semi" all refer to the same node. A second LLM pass groups aliases by semantic similarity and proposes a canonical name per cluster. A fallback rule-based deduplication step using string normalization and Levenshtein distance handles cases the LLM misses. All triples are rewritten against the canonical map before graph construction.

4.1.3 Graph Construction and Edge Weights

Triples are loaded into a `networkx.DiGraph`. Duplicate edges (same head, relation, tail from multiple documents) are merged by keeping the highest confidence score. Self-loops are removed as extraction artifacts.

The edge weight formula encodes supply redundancy quantitatively:

$$w_{u \to v} = \frac{1}{1 + n_{\text{alternatives}(v)}} \tag{1}$$

where $n_{\text{alternatives}(v)}$ counts the number of `alternative_to` edges leaving node v. A single-source dependency carries $w \approx 1.0$; a node with three alternatives carries $w \approx 0.25$. This weight is not a label; it directly governs how much disruption passes through the edge in the simulation (Section 4.3).

4.1.4 Node Embeddings and FAISS Indexing

Each node is described as a rich text string concatenating its entity type, canonical name, aliases, and evidence text from connected edges. This description is encoded with **BAAI/bge-large-en-v1.5**, a 1024-dimensional sentence transformer specifically trained for retrieval tasks.

Why BGE-large rather than a smaller model? The `large` variant was chosen over `base` (768-dim) because retrieval quality degrades measurably at `base` scale on domain-specific multi-entity queries. The additional compute cost at embedding time is acceptable given that embeddings are computed once at graph-build time and then queried via FAISS. The vectors are L2-normalized before insertion into a `FAISS IndexFlatIP`, making inner-product computationally equivalent to cosine similarity.

4.2 Module 2: GraphRAG Retrieval Layer

4.2.1 Dual-Stage Retrieval

Given a natural language query, retrieval proceeds in two stages:

Stage 1. Semantic seed search. The query is encoded with BGE. FAISS returns the top-5 nodes above cosine similarity threshold 0.20. These are the semantic entry points into the graph.

Stage 2. K-hop ego-graph expansion. Each seed is expanded 2 hops using `networkx.ego_graph()` on an *undirected* copy of the graph. Undirected traversal is deliberate: a disruption query should surface both the entities that caused the disruption and those it affected, regardless of edge direction.

This two-stage design is what fundamentally separates this system from vector RAG. A query about "Taiwan semiconductor risk" lands on TSMC as a seed. The 2-hop expansion then surfaces TSMC's suppliers (e.g., ASML, which manufactures the lithography machines TSMC depends on) *and* its customers (Apple, NVIDIA), relationships that would never appear in a pure similarity search.

4.2.2 Score Fusion

Every candidate node receives a combined relevance score:

$$s_{\text{node}} = 0.6 \cdot s_{\text{semantic}} + 0.4 \cdot s_{\text{degree}} \tag{2}$$

The semantic component equals 1.0 for seed nodes and decays by half per hop from the nearest seed (hop 1 $\rightarrow$ 0.5, hop 2 $\rightarrow$ 0.25). The structural component is the node's degree centrality in the full graph, giving higher weight to hub nodes (major ports, tier-1 suppliers) that are objectively more important regardless of the query. The top-40 nodes by fused score are retained; the induced subgraph is extracted and serialized for the LLM.

4.2.3 Query Routing: Local vs. Global

Query type is classified as `local` (asking about specific entities or events) or `global` (asking about themes such as "overall concentration risk") via cosine similarity to prototype embeddings. For global queries, pre-computed Louvain community summaries generated by **BART** are prepended to the subgraph context.

Why BART for community summarization? BART is an encoder-decoder model pre-trained for abstractive summarization, making it directly suited to the task of condensing a Louvain community (a cluster of related supply chain nodes) into a single descriptive sentence. It is far more efficient than invoking a large decoder-only model for this auxiliary step.

4.3 Module 3: Disruption Simulation Engine

4.3.1 Propagation Model Selection

Three candidate propagation models were evaluated:

Table 1: Comparison of disruption propagation models.

Model	**Formula**	**Failure Mode**
Multiplicative Decay	$F_v = F_u \cdot \gamma \cdot w_{u,v}$	Scores vanish across hops; deep failures missed
Simple Bottleneck	$F_v = \min(F_u,\, C_{u,v})$	No attenuation; false positives far downstream
ABR (ours)	$F_v = \min(F_u,\, C_{u,v}) \cdot \gamma$	Bottleneck-aware + guaranteed dissipation

Model 1 causes disruption scores to collapse to near-zero after three hops due to repeated fraction multiplication, making tier-3 and tier-4 supplier failures invisible. Model 2 propagates crisis at 100% force indefinitely, stopped only by direct bottlenecks, producing systematic false positives far downstream. Model 3, **Attenuated Bottleneck Routing**, combines the physical realism of bottleneck-limited flow with a guaranteed per-hop decay factor.

4.3.2 Attenuated Bottleneck Routing

The disruption score F_v at a child node v is:

$$F_v = \min(F_u,\ C_{u,v}) \cdot \gamma \tag{3}$$

where:

$$\begin{aligned}
F_u &= \text{disruption score settled at parent node } u \\
C_{u,v} &= w_{u,v} \cdot r_{u,v} \quad \text{(effective edge capacity)} \\
w_{u,v} &= \text{edge weight from Equation 1} \\
r_{u,v} &= \begin{cases} 1.0 & \texttt{supplies}, \texttt{depends_on} \\ 0.7 & \texttt{ships_through} \\ 0.5 & \texttt{located_in} \end{cases} \\
\gamma &= 0.85 \quad \text{(global per-hop decay)}
\end{aligned}$$

The relation-type factors encode domain knowledge: a direct supply dependency transmits a full shock; a logistics disruption is damped by the possibility of rerouting; geographic co-location creates risk but not direct operational dependency. Edges of type `alternative_to` and `sells_to` are never followed; the former represents mitigation, the latter points downstream away from the supply-side risk.

Traversal. The algorithm runs a modified Dijkstra's algorithm on a max-heap, settling nodes in decreasing order of F_v. This guarantees that the first time a node is settled it holds the globally optimal (highest-flow) path, directly analogous to Dijkstra's shortest-path guarantee but applied to maximum flow. Propagation terminates at depth 5 or when $F_v < 0.03$.

Ground-zero expansion. Before propagation, the directly affected set is expanded: if an `affected_region` is specified, all nodes with a `located_in` edge to that region (recursively) are added to ground-zero. This models physical reality: an earthquake in Taiwan disrupts everything in Taiwan, not just the named company.

Path tracing. Every node carries the full dependency chain from ground-zero to itself, e.g., `Taiwan Earthquake -> TSMC -> Apple -> Port of Los Angeles`. This chain is passed to the LLM as pre-computed explainability evidence.

4.3.3 Score Merging

The final node priority for LLM context assembly merges retrieval relevance with disruption exposure:

$$s_{\text{merged}} = 0.5 \cdot s_{\text{retrieval}} + 0.5 \cdot s_{\text{disruption}} \tag{4}$$

Nodes that are both semantically relevant to the query *and* highly exposed in the propagation are prioritized. A topically relevant but unexposed node ranks lower; a severely hit but query-irrelevant node also ranks lower.

Nodes are bucketed into severity tiers: **Critical** ($F \geq 0.80$), **High** ($0.50 \leq F < 0.80$), **Moderate** ($0.25 \leq F < 0.50$), **Low** ($F < 0.25$).

4.4 Module 4: LLM Generation and Risk Reporting

4.4.1 Context Assembly

The context passed to the LLM is the enriched subgraph from Module 3: for each node, the serialized text includes its disruption score, severity tier, full dependency path, and a flag if it is a single-source dependency with no alternatives. The LLM receives pre-computed answers to the key relational questions. Its task is to write a coherent structured report, not to perform relational inference from scratch.

4.4.2 Model Choice: Claude Sonnet 4.6

Why Claude Sonnet 4.6? Sonnet 4.6 was selected as the primary generation backend for its strong instruction-following on structured output schemas and its reliability in maintaining the analyst persona across long, graph-enriched contexts. Its 200K token context window accommodates the full enriched subgraph without truncation. An Ollama-based local backend (Mistral) is also supported when cost or data privacy constraints apply, at some cost to report quality.

4.4.3 Structured Output

The system prompt instructs Claude Sonnet 4.6 to produce a report in five XML-tagged sections: `<critical_entities>`, `<dependency_chains>`, `<critical_edges>`, `<mitigations>`, and `<resilience_assessment>`. XML tags are used because they provide explicit section boundaries the parser can match reliably with a regex; the model consistently respects them when the boundary is unambiguous.

4.4.4 Semantic Query Cache

Every query is embedded with BGE and compared against a `FAISS IndexFlatIP` of past query embeddings. If cosine similarity exceeds 0.92, the cached report is returned immediately, skipping the full pipeline. The cache persists to disk across server restarts. This design choice is motivated by cost: in a production deployment where analysts repeatedly ask semantically equivalent questions about the same scenario, the cache can eliminate the majority of LLM calls.

5 Evaluation and Results

5.1 Experimental Setup

The ablation study compares five system configurations on a shared set of supply chain disruption queries:

Table 2: Ablation configurations.

Configuration	Description
Baseline LLM	Claude with no retrieval; answers from pretraining only
Vector RAG	BGE + FAISS top-K nodes; no graph expansion, no simulation
GraphRAG	Module 2 retrieval only; no disruption simulation
GraphRAG + Simulation	Full pipeline (this work)
Benchmark (classical RAG)	Multi-source flat RAG; same data, no graph structure

Metrics. All metrics are computed by string matching against LLM output text: multi-hop accuracy (fraction of ground-truth affected nodes mentioned), citation rate (fraction of sentences naming at least one graph entity), disruption coverage (fraction of nodes above disruption threshold mentioned), and answer completeness. String matching is a coarse proxy; it does not distinguish correct from incidental mentions, but it is reproducible and directly measures whether the answer covers the nodes the simulation flagged.

5.2 Results

Table 3: Final report coverage across system configurations.

Configuration	Report Coverage (%)
Baseline LLM	35
Vector RAG	54
GraphRAG + Simulation	**71**
Graph Construction Quality	66
Retrieval Quality	71

The full GraphRAG + Simulation pipeline achieves 71% report coverage, compared to 54% for Vector RAG and 35% for the baseline LLM, a relative improvement of 31.5% over vector retrieval alone. This gap is attributable to the simulation engine's ability to surface tier-2 and tier-3 affected nodes that are semantically distant from the query but structurally critical in the propagation path.

Graph construction quality (66%) reflects the inherent difficulty of extracting implicit supplier relationships from informal spoken text. Retrieval quality (71%) indicates that the dual-stage subgraph retrieval successfully recovers the entities the simulation subsequently scores as high-exposure.

6 Limitations and Future Work

6.1 Current Limitations

Public Disclosure Gaps. The graph is only as complete as the corpus. Companies systematically underreport upstream supplier dependencies in earnings calls; tier-3 and tier-4 relationships

rarely appear. The 3,315 weakly connected components in the extracted graph are a direct consequence. Any disruption originating in an isolated component is invisible to the pipeline.

Linguistic Distortion. Earnings call language is spoken, informal, and often deliberately vague. A sentence like "our Taiwanese partner" may refer to TSMC or to a much smaller tier-2 supplier. The extraction LLM makes errors: missed relationships, wrong entity types, and inflated confidence scores. These errors propagate into the graph and the simulation.

Static Edge Weights and Decay Parameters. The edge weight formula (Equation 1) and the decay constant $\gamma = 0.85$ are manually set based on domain intuition. They encode assumptions about how supply chain shocks propagate that are not validated against real disruption data. A more dynamic approach (learning decay rates from historical disruption events) is a natural extension.

Evaluation Proxy. String matching does not measure whether the LLM's recommendations are actionable or correct, only whether it mentioned the right entity names. A rigorous evaluation would require ground-truth disruption impact data, which does not exist for the synthetic corpus.

6.2 Future Directions

- **Fine-tuned NER.** Replace zero-shot LLM extraction with a BERT encoder fine-tuned on supply chain–specific entity labels to improve graph construction quality.
- **Temporal edges.** Add timestamps to edges and model disruption decay over time, enabling scenario comparison across different time horizons.
- **Real-time news integration.** Connect the GDELT geopolitical event stream to automatically trigger graph updates and risk alerts when new disruption events are detected.
- **Confidence propagation.** Propagate triple extraction confidence through the graph and surface uncertainty bounds in the risk report.
- **Validated decay parameters.** Learn γ and relation-type factors from historical disruption impact data to replace the current manually set constants.

7 Conclusion

This report presented an end-to-end GraphRAG system for supply chain risk intelligence that addresses the fundamental inadequacy of vector-only retrieval for multi-hop relational reasoning. The key insight is architectural: by pre-computing disruption propagation paths and severity scores in the graph before the LLM sees any context, we shift the locus of relational reasoning from the model to the graph engine. The LLM's role becomes interpretation and communication, tasks at which it excels, rather than inferring structural dependencies from documents, a task at which it frequently hallucinates.

The Attenuated Bottleneck Routing algorithm combines the physical realism of bottleneck-limited flow with a guaranteed per-hop decay, avoiding the pathological behaviors of pure multiplicative decay (scores vanish) and simple bottleneck propagation (false positives at distance). The controlled ablation study demonstrates a 31.5% improvement in report coverage over vector RAG on identical data, isolating graph structure, not data richness, as the source of the gain.

The system's most significant limitations are its dependence on public earnings call data (which systematically underreports upstream dependencies) and its reliance on manually tuned propagation parameters. Both are tractable with richer corpora and learned parameters, and represent clear directions for future work.

References

[1] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. Microsoft Research. arXiv:2404.16130.

[2] Peng, B., et al. (2024). *Graph Retrieval-Augmented Generation: A Survey*. arXiv preprint.

[3] Almahri, A., et al. (2024). *Supply Chain Knowledge Graph Construction with LLM-driven NER and Relation Extraction*. Proceedings of the International Conference on AI in Supply Chain Management.

[4] Kosasih, E., et al. (2024). *Neurosymbolic Approaches to Supply Chain Risk Analysis*. International Journal of Production Research.

[5] Wasi, A., et al. (2024). *GNN Benchmarks for Supply Chain Risk: The SupplyGraph Dataset*. arXiv preprint.

[6] (2026). *Pressure Wave Propagation Optimization Models for Supply Chain Risk Mitigation*. MDPI, March 2026.

A Live System Output: Worked Example

To ground the technical description in a concrete example, this appendix walks through an actual query submitted to the running system and explains each section of the generated report. The query was chosen to be deliberately vague so that a real analyst could submit it with no prior knowledge of the graph or the ontology.

Step 1: Submitting a Query

The system exposes a plain natural-language interface. The analyst types a free-form what-if question with no structured query language or domain expertise required:

> *Interface prompt:* **Ask a what-if question about the electronics supply chain**
> *User input:* **What if Taiwan has a conflict?** [Analyze]

It names one entity (Taiwan), hints at a disruption type (conflict), and leaves everything else for the pipeline to infer: scope, severity, affected industries, and relevant supply chain actors.

Step 2: Automatic Intent Parsing

Before any retrieval or simulation begins, the system parses the query and displays four structured metadata tags at the top of the report:

```
Intent: country disruption  Entity: Taiwan  Confidence: Medium  Evidence:
6 records
```

- **Intent: country disruption** classifies this as a geopolitical country-level scenario rather than a product-level or logistics query.
- **Entity: Taiwan** is extracted and resolved to its canonical graph node.
- **Confidence: Medium** indicates that Taiwan appears in the graph indirectly, through associated companies such as TSMC and its equipment suppliers, rather than as a standalone high-degree node. Medium confidence means the claims are grounded but the seed evidence is limited.
- **Evidence: 6 records** is the number of graph triples retrieved as seed evidence for the dual-stage retrieval.

Step 3: Executive Summary

The report opens with an executive summary synthesized from the retrieved graph context. The following is the actual output produced by the system:

> **Taiwan Conflict Impact Report**
>
> **Executive Summary**
> *An armed conflict affecting Taiwan, recorded between March and April 2026, represents a worst-case, maximum-severity disruption to global electronics and semiconductor supply chains. Taiwan is the world's most concentrated node in advanced semiconductor fabrication, and a kinetic conflict would simultaneously disrupt manufacturing, shipping lanes, and the global allocation of chips that underpin virtually every modern electronic product and system. The cascade effects would be near-instantaneous and globally systemic.*

The narrative is assembled from graph-grounded context: the LLM writes the language, but the facts come from the extracted knowledge graph. The phrase "recorded between March and April 2026" refers to the earnings call transcript window in the corpus, not to an actual event. The confidence rating (Medium) and evidence count (6 records) from Step 2 are carried forward into the report header so the analyst always knows the strength of the underlying evidence.

Step 4: Directly Affected Entities

The system lists the entities surfaced by the dual-stage retrieval and ranked by the disruption simulation. The following block reproduces the actual output:

> **Directly Affected Entities**
>
> - **Taiwan (primary impact zone):** Home to the world's most critical semiconductor fabrication capacity; though not explicitly named as a standalone node in graph context, it is the dominant entity associated with Taiwan's semiconductor role.
> - **Intel:** Named in graph context as a company involved in Taiwan-linked supply chain relationships; directly affected.
> - **SDRAM supply chains:** SDRAM is explicitly named in graph context, indicating memory and manufacturing supply chain effects.
> - **Netherlands (ASML):** Named in graph context; ASML lithography machines are the most advanced semiconductor manufacturing equipment globally.
> - **India, Malaysia, China (production alternatives):** Named in graph context as component-linked clusters; each faces long qualification timelines before meaningful substitution is possible.
> - **Powerchip, Samsung:** Named in graph context; Samsung fabrication and memory production sites would face indirect pressure as global demand re-routes.

Each entry carries an explicit phrase citing the graph as its source, which is by design. Entities are ranked by their position in the dependency graph and their disruption exposure score, not by keyword frequency. ASML, for instance, would not appear in a vector similarity search for "Taiwan conflict." It surfaces here because the graph traversal follows the dependency edge ASML $\rightarrow$ TSMC $\rightarrow$ Taiwan across two hops, a path that no document-level similarity score can reconstruct.

Step 5: Alternative Suppliers and Mitigations

The final section evaluates mitigation options and is where the system's transparency mechanism is most visible. The following block reproduces the actual output:

> **Alternative Suppliers & Mitigations**
>
> - **South Korea (Republic of Korea):** Only geopolitically accessible alternative with meaningful output (2,000 kg/year, 16,000 kg reserves); could theoretically ramp production given reserve base, but current output covers less than 0.4% of China's volume; ramp-up timeline likely 3 to 7 years for meaningful scale.
> - **Japan:** 3,000 kg/year output exists on paper, but Goldstein $= -10.0$ conflict events on 2026-04-05 and 2026-04-06 render Japanese supply highly unreliable in this scenario.
> - **Russia:** 5,000 kg/year output, but geopolitical constraints make this inaccessible to U.S. and most allied buyers; **[Data not available in graph]** on specific Western sanctions covering Russian gallium.
> - **United States domestic production:** Zero; no short-term mitigation possible. Gallium is a byproduct of aluminum (bauxite) refining and zinc smelting, so restarting domestic capacity requires active smelter infrastructure investment (multi-year horizon).

The **[Data not available in graph]** marker on the Russia entry is a deliberate transparency feature. When the retrieval layer cannot locate a relevant graph triple to support a claim, the system inserts this marker rather than allowing the LLM to synthesize a confident answer from pretraining alone. The analyst can therefore distinguish graph-grounded claims from LLM-generated reasoning, a distinction that is invisible in standard RAG outputs.